

Infrastructure as Code

Become a wizard...

This could be you...

In your instructor's opinion...

- › **Infrastructure as Code (IaC), e.g. Terraform is a superpower!**
- › Software Developers used to only create software. Someone else (Infra team) had to (manually) provision infrastructure
- › Now, you, the DevOps Data Engineer gets to create/terraform your own world!
- › But, with great power comes great responsibility...



Creating resources manually is error prone. In the cloud we can use tools like terraform to deploy resources based on code, thereby significantly simplifying the setup.

Old world: long, error prone, outdated, manuals..

1. Sign in to AWS Management Console:

1. Go to AWS Management Console.
2. Sign in with your credentials.

2. Navigate to the S3 Service:

1. In the AWS Management Console, search for "S3" in the search bar.
2. Click on "S3" under the Services dropdown.

3. Create a New S3 Bucket:

1. In the S3 dashboard, click on the "Create bucket" button.
2. Enter a unique bucket name in the "Bucket name" field.
3. Select the AWS Region where you want the bucket to be created (e.g., us-west-2).

4. Configure Bucket Settings: ...

New world: infrastructure as code

```
provider "aws" {  
  region = "us-west-2"  
}  
  
resource "aws_s3_bucket" "my_bucket" {  
  bucket = "my-unique-bucket-name"  
  acl    = "private"  
}  
  
output "bucket_name" {  
  value = aws_s3_bucket.my_bucket.bucket  
}
```

What is Infrastructure as Code?

Infrastructure as Code (IaC) is a key practice in modern IT and cloud computing that involves managing and provisioning computing infrastructure through machine-readable configuration files, rather than through physical hardware configuration or interactive configuration tools. By treating infrastructure the same way developers treat application code, IaC allows organizations to automate the deployment and management of their IT environments with precision, repeatability, and consistency. This approach fundamentally transforms how infrastructure is managed, enabling greater agility, efficiency, and control. Some key aspects and benefits of Infrastructure as Code:

- › **Automation and Consistency:** One of the primary benefits of IaC is the automation of infrastructure provisioning. Instead of manually configuring servers, networks, and other components, IaC uses scripts and configuration files to automate the entire process. This ensures consistency across environments, as the same code can be used to replicate infrastructure in multiple environments (development, testing, production) without the risk of human error. This consistency is crucial in reducing bugs and deployment failures that stem from environment discrepancies.
- › **Version Control and Documentation:** With IaC, infrastructure configurations are stored in version-controlled repositories, just like application code. This allows teams to track changes over time, roll back to previous versions if something goes wrong, and collaborate more effectively on infrastructure management. It also provides a clear and documented history of the infrastructure setup, making it easier to audit, understand, and maintain over time.

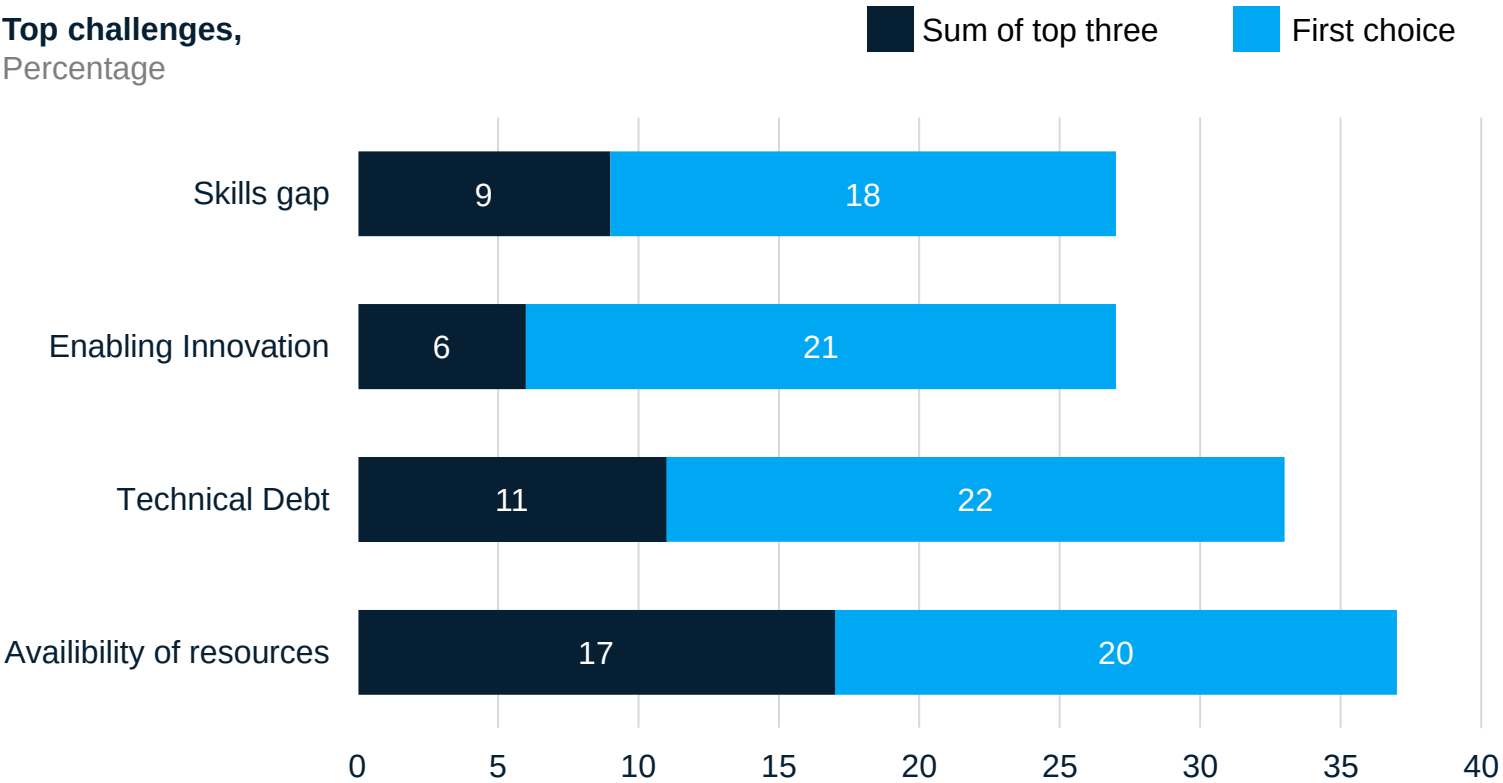
What is Infrastructure as Code?

- › **Scalability and Flexibility:** IaC is particularly powerful in cloud computing environments, where infrastructure needs to scale rapidly in response to demand. With IaC, organizations can quickly spin up new instances, deploy complex environments, or tear down unused resources, all through automated scripts. This scalability is essential in dynamic cloud environments, where resources need to be managed efficiently to optimize costs and performance.
- › **Reduction of Drift and Enhanced Testing:** Configuration drift, where the actual state of infrastructure diverges from the intended state, is a common problem in traditional IT management. IaC mitigates this issue by ensuring that infrastructure can be consistently recreated from the codebase, preventing unintentional changes and misconfigurations. Additionally, IaC enables infrastructure to be tested in the same way as application code, through unit tests, integration tests, and continuous integration/continuous deployment (CI/CD) pipelines. This leads to more reliable and robust infrastructure setups.
- › **Cost Efficiency and Resource Optimization:** By automating infrastructure management and enabling precise control over resource allocation, IaC helps organizations optimize their cloud spending. Resources can be provisioned on-demand and scaled back when not needed, reducing waste and ensuring that IT budgets are used effectively. This level of control is especially important in cloud environments where costs can escalate quickly if not properly managed.
- › **Enhanced Security and Compliance:** IaC also plays a crucial role in security and compliance. With infrastructure managed through code, security policies, access controls, and compliance requirements can be embedded directly into the infrastructure code, ensuring they are consistently applied across all environments. This reduces the risk of security breaches and helps organizations maintain compliance with industry regulations more easily.

IaC Skills are high in demand and one of the bottlenecks in the industry

Top Challenges/Threats Faced by IT I&O Organizations

Top challenges,
Percentage



Footnote: 1. Q: What are the top 3 challenges or threats that your IT Infrastructure and Operation organization face currently? 2023 Gartner I&O Leaders Signature Role Survey

Source: <https://www.gartner.com/doc/reprints?id=1-2GV6N7OV&ct=240308&st=sb>

Conclusion

- Infrastructure automation and orchestration (IA&O) evolved to improve speed, quality, and reliability in system deployment and management, integrating complex workflows and tools.
- DevOps, platform engineering, and I&O teams use IA&O tools for scalable, autonomous, and reliable infrastructure delivery and operations.
- The IA&O market is growing as demands on infrastructure teams increase due to rapidly changing business and technology needs
- Key challenges for I&O teams include resource availability, technical debt management, enabling innovation, and skills proficiency.

AWS CloudFormation and Azure Resource Manager (ARM) templates are native services provided by cloud providers to automate and manage infrastructure resources declaratively.

AWS Cloud Formation



- › Service Scope: Automates the provisioning and management of AWS resources using declarative YAML/JSON templates.
- › Features:
 - › Supports stack management with rollback and drift detection capabilities.
 - › Tight integration with all AWS services, including serverless and containerized applications.
 - › Enables resource dependency tracking and efficient orchestration.
 - › Offers a StackSets feature to manage resources across multiple AWS accounts and regions.

Azure Resource Manager Templates



- › Service Scope: Automates the deployment, configuration, and management of Azure resources using JSON or Bicep templates.
- › Features:
 - › Built-in integration with Azure Resource Groups for logical resource organization.
 - › Supports modular and reusable templates through linked templates and parameters.
 - › Provides declarative deployment with resource dependency management.
 - › Offers a preview of changes before deployment using the "What-If" analysis.

Terraform and Pulumi are both infrastructure-as-code tools; Terraform uses its own declarative language, HCL, while Pulumi allows the use of GPP languages like Python,

Terraform



- › Terraform is an open-source infrastructure as code (IaC) tool created by HashiCorp. It allows users to define and provision data center infrastructure using a high-level configuration language, called Hashi Corp Language (HCL). It supports multiple cloud providers, making it a popular choice for managing infrastructure across different environments.
- › **Pros:** Wide Adoption, Multi-Cloud Support, Mature and Stable
- › **Cons:** Steep Learning Curve, Limited Flexibility, State Management Complexity

Pulumi



- › Pulumi is a modern IaC tool that allows users to write code in general-purpose programming languages (e.g., TypeScript, Python, Go) to define and manage infrastructure. It integrates with multiple cloud providers and aims to bring more flexibility and expressiveness to IaC.
- › **Pros:** Flexibility, Strong Developer Integration, Rich Ecosystem
- › **Cons:** Less Mature, Complexity for Simple Tasks, Vendor Lock-In

Terraform Basics

Basic terraform commands

There are a handful of basic terraform commands:

- › `terraform init`
- › `terraform validate`
- › `terraform plan`
- › `terraform apply`
- › `terraform destroy`

You can learn about the version you are using:

- › `terraform -version`

You can find more information using help:

- › `terraform -help`

HashiCorp Configuration Language (HCL)

```
# Template
<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {
  # Block body
  <IDENTIFIER> = <EXPRESSION> # Argument
}

# AWS EC2 Example
resource "aws_instance" "web_server" { # BLOCK
  ami = "ami-04d29b6f966df1537"
  instance_type = var.instance_type
}
```

The basic terraform workflow is: Write, Plan, Apply!

Write

- Author infrastructure as code using HCL or JSON.
- Define providers, resources, variables, outputs, and modules.
- Make configurations modular and reusable.

Plan

- Initialize the working directory with terraform init.
- Preview changes with terraform plan.
- Review the execution plan to understand the impact of changes.

Apply

- Apply the changes with terraform apply.
- Confirm the execution plan and provision the infrastructure.
- Terraform ensures the desired state is achieved and updates the state file.

The **provider** block is used to configure the providers that Terraform will use to interact with the various infrastructure platforms

Overview

- › In Terraform, a **provider block** is used to configure the providers that Terraform will use to interact with the various infrastructure platforms. Providers are responsible for understanding the API interactions with the services and exposing the resources within those services to Terraform.
- › Each provider is a plugin that Terraform uses to manage and interact with a particular service or platform. Providers can be for cloud services (like AWS, Azure, Google Cloud), as well as for local systems (like local file systems, and Docker).
- › The provider block abstracts the interaction with various platforms, making it easier to manage resources across different services. It centralizes the configuration for interacting with external services, making it easier to manage and update.

Template & Example

```
provider "provider_name" {  
    # Configuration options (arguments) for the provider  
}  
  
provider "aws" {  
    region = "us-west-2"  
    profile = "my-aws-profile" # Optional  
}  
  
provider "local" {  
}  
  
provider "docker" {  
}
```

The resource block is used to define and manage infrastructure components

Overview

- › In Terraform, a **resource block** is used to define and manage infrastructure components. Resources are the basic building blocks of your infrastructure and can represent physical or virtual components.
- › **Declarative:** You declare the desired state of your infrastructure, and Terraform ensures that this state is achieved.
- › **Idempotent:** Running the same configuration multiple times will produce the same result, making it safe to apply changes incrementally.
- › **Composable:** Resources can be easily composed and combined to build complex infrastructure setups.
- › **Reusability:** Resource configurations can be modularized and reused across different environments or projects.

Template & Example

```
# AWS Provider
resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbfafa1f0"
  instance_type = "t2.micro"
  tags = {
    Name = "example-instance"
  }
}

# Local provider
resource "local_file" "example" {
  content  = "Hello, Terraform!"
  filename = "${path.module}/hello.txt"
}

# Docker provider
resource "docker_image" "nginx" {
  name = "nginx:latest"
}
```

The `locals` block in Terraform defines named local values/variables within a configuration, enabling reuse, and simplification of expressions across the code.

Overview

- › In Terraform, **local variables** are a way to simplify and reduce repetition in your configuration by allowing you to define reusable expressions or values that can be referenced multiple times within a module.
- › **Readability:** Local variables make your configuration easier to read and understand by centralizing complex expressions and repeated values.
- › **Maintainability:** Simplifies updates to your configuration since you only need to change a value in one place.
- › **Reusability:** Local variables can be used to define reusable expressions that can be referenced multiple times within a module.
- › **Consistency:** Ensures consistency by reducing the risk of typos or mismatched values when the same value is used in multiple places.

Template & Example

```
locals {  
  # Block body  
  local_variable_name = <EXPRESSION OR VALUE>  
  local_variable_name = <EXPRESSION OR VALUE>  
}
```

```
locals {  
  ami_id = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "example-instance"  
    Environment = "development"  
  }  
}
```

The **data** block is used to fetch and reference information that is defined outside of Terraform's configuration or managed by another entity

Overview

- › In Terraform, a **data block** is used to fetch and reference information that is defined outside of Terraform's configuration or managed by another entity. This is particularly useful for retrieving information about existing infrastructure components, such as AWS AMIs, security groups, networking details, and more, without creating or modifying those components.
- › **Reusability:** Makes your Terraform configurations more reusable by dynamically fetching necessary information.
- › **Consistency:** Helps ensure consistency by using information about existing infrastructure rather than duplicating configurations.
- › **Flexibility:** Provides the ability to adapt to changes in the external environment without modifying your Terraform configurations.

Template & Example

```
data "<DATA TYPE>" "<DATA LOCAL NAME>" {
  <IDENTIFIER> = <EXPRESSION> # Argument
}

provider "aws" {
  region = "us-west-2"
}

data "aws_ami" "latest_amazon_linux" {
  most_recent = true
  filter {
    name      = "name"
    values    = ["amzn-ami-hvm-*-x86_64-gp2"]
  }
  filter {
    name      = "virtualization-type"
    values    = ["hvm"]
  }
  owners     = ["137112412989"]
}
```

Input variables are used to provide configurable values to your Terraform configurations.

Overview

- › In Terraform, **input variables** are used to provide configurable values to your Terraform configurations. They allow you to parameterize your configurations and make them more flexible, reusable, and easier to manage. Input variables can be defined in your configuration files and their values can be provided in various ways, such as through command-line arguments, environment variables, or separate variable definition files.
- › **Reusability:** Make your configurations more reusable by parameterizing values that might change.
- › **Flexibility:** Easily adjust configurations for different environments (e.g., development, staging, production).
- › **Maintainability:** Simplify the management of configuration changes by centralizing variable definitions.

Template & Example

```
variable "<VARIABLE_NAME>" {  
  # Block body  
  type = <VARIABLE_TYPE>  
  description = <DESCRIPTION>  
  default = <EXPRESSION>  
  sensitive = <BOOLEAN>  
  validation = <RULES>  
}
```

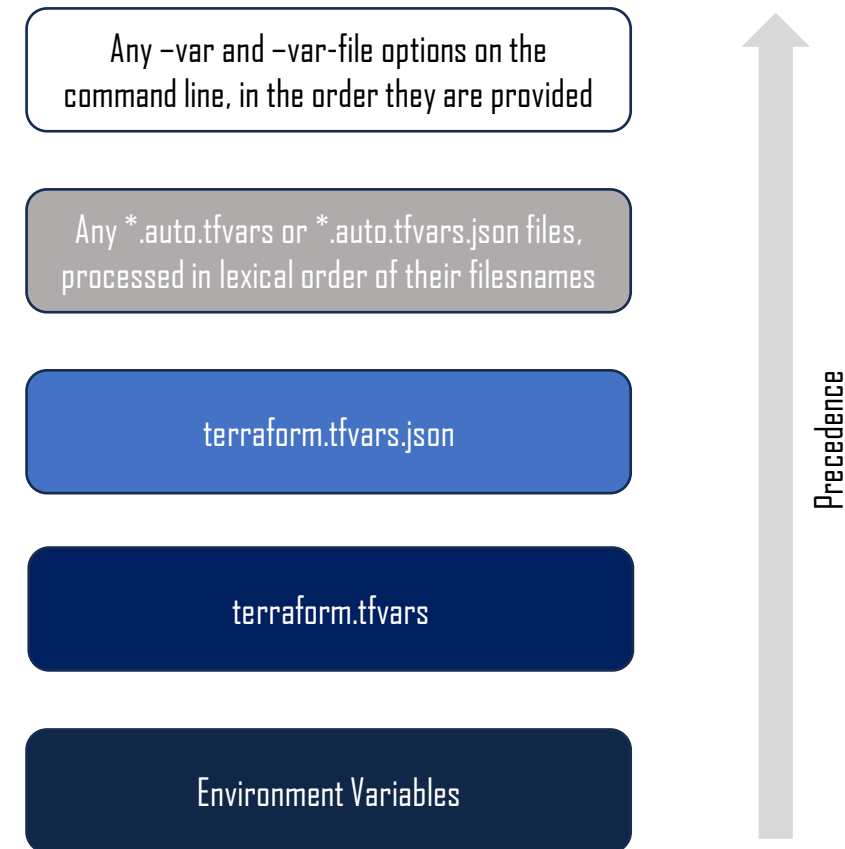
```
variable "instance_count" {  
  description = "The number of EC2 instances to create"  
  type        = number  
  default     = 1  
  validation {  
    condition = var.instance_count <= 10  
    error_message = "instance_count must be < 10"  
  }  
}
```

In `.tfvars` files values of variables can be stored which are expected to change in different environments or to separate them logically

Overview

- › Files that end on `.tfvars` are automatically understood by terraform as files which contain variable values
- › `*.auto.tfvars` are automatically processed in lexical order of their file names.
- › Typical pattern, one file per environment:
`lab.auto.tfvars`, `tst.auto.tfvars`, `acc.auto.tfvars`,
`prd.auto.tfvars`

Priority order of variables in terraform



Modules are a way to encapsulate, and reuse sets of resources across different parts of your configuration

Overview

- › In Terraform, **modules** are a way to encapsulate, and reuse sets of resources across different parts of your configuration. A module is essentially a container for multiple resources that are used together. It can be thought of as a function in programming, where you define the logic once and then call it multiple times with different parameters.
- › **Reusability:** Modules allow you to reuse configurations across different projects or environments.
- › **Maintainability:** Encapsulating related resources in a module makes it easier to manage and update your infrastructure.
- › **Abstraction:** Modules provide a level of abstraction, hiding the complexity of resource configurations and exposing simple interfaces for configuration.

Template & Example

```
module "<MODULE_NAME>" {  
  # Block body  
  source = <MODULE_SOURCE>  
  <INPUT_NAME> = <DESCRIPTION> #Inputs  
  <INPUT_NAME> = <DESCRIPTION> #Inputs  
}  
  
module "my_ec2_instance" {  
  source          = "./ec2_instance"  
  ami_id          = "ami-0c55b159cbfafa1f0"  
  instance_type   = "t2.micro"  
  tags = {  
    Name = "example-instance"  
  }  
}
```

The output block is used to define values that will be displayed to the user or exported for use by other Terraform configurations

Overview

- › In Terraform, the **output block** is used to define values that will be displayed to the user or exported for use by other Terraform configurations. Outputs are particularly useful for retrieving information about the resources you create, such as IP addresses, instance IDs, or any other relevant data that you might need to reference or use elsewhere.
- › **Information Sharing:** Outputs provide a way to share information about your resources, both with users and other Terraform configurations.
- › **Modularity:** Outputs can be used to pass information between modules, making your configurations more modular and reusable.
- › **Debugging:** Outputs can help with debugging by showing the values of specific resources after terraform apply is run.

Template & Example

```
output output_name {  
  value      = <EXPRESSION>  
  description = <DESCRIPTION> # Optional  
  sensitive  = <BOOLEAN>     # Optional  
  depends_on = [<RESOURCE>]  # Optional  
}
```

```
output "instance_id" {  
  value      = aws_instance.example.id  
  description = "The ID of the created EC2 instance"  
}
```

Apply complete! Resources:
1 added, 0 changed, 0 destroyed.

Outputs:
instance_id = i-0abcdef1234567890

Modules are a way to encapsulate, and reuse sets of resources across different parts of your configuration

Structure of a module

In `root/main.tf` you "initialize/call" the module with a path to its (e.g., `root/modules/ec2_module`). All tf code of the module go into its folder (`root/modules/ec2_module`). The following structure is typical

- › **main.tf**: In this file (and possibly others, define all the resources of the module)
- › **variables.tf**: Defines all variables that are inputted into the module (e.g., VPN IDs or security group ARNs)
- › **outputs.tf**: Defines all outputs that the module returns (e.g., IP addresses)

```
module "my_ec2_instance" {  
  source      = "./modules/ec2_module"  
  ami_id      = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
  tags = ...  
}
```

Typical Structure of a terraform repo with modules

```
terraform-root  
|--- modules  
|   |--- ec2_module  
|   |   |-- main.tf  
|   |   |-- variables.tf  
|   |   |-- outputs.tf  
|   |--- vpc_module  
|   |   |-- ...  
|   |--- subnet_module  
|   |   |-- ...  
|--- main.tf  
|--- variables.tf  
|--- outputs.tf
```

Terraform repository strategies

Structure of a module

- › One repository per environment: Easier to handle differences in each environment, but more difficult to keep in sync. Usually not the best idea.
- › One `<env>.tfvars` file per environment and resources only required in a particular environment are declared as such:

```
resource "aws_s3_bucket" "lab_bucket" {  
    count = var.environment == "lab" ? 1 : 0  
    bucket = "my-lab-bucket-${random_pet.suffix.id}"  
    acl    = "private"  
}
```

Strategy with a tfvars file per env

```
terraform-root/  
├── modules/  
│   ├── ec2_module/  
│   │   ├── main.tf  
│   │   ├── variables.tf  
│   │   └── outputs.tf  
│   ├── vpc_module/  
│   │   ├── main.tf  
│   │   ├── variables.tf  
│   │   └── outputs.tf  
│   └── subnet_module/  
│       ├── main.tf  
│       ├── variables.tf  
│       └── outputs.tf  
├── main.tf  
├── variables.tf  
├── outputs.tf  
├── dev.tfvars  
├── tst.tfvars  
├── acc.tfvars  
└── prd.tfvars
```

Code Demo

Find the code demo here: <https://gitlab.com/dsti-data-pipeline-2/gitlab-commit-cheater>

Windows Installation instructions:

<https://app.tango.us/app/workflow/Downloading-Terraform-on-Windows--A-Quick-Tutorial-63634416f09348c4857f64e3804235a2>